Campus San Alberto Hurtado, S.J. de Quetzaltenango
Facultad de Ingeniería
Ingeniería en Informática y Sistemas

Laboratorio No. 2 - Manejo de archivos

Agenda de Contactos con Acceso Aleatorio

Sebastián García Morales
Carné: 1562025

Quetzaltenango, noviembre de 2026

1. Descripción de la aplicación

La aplicación es una agenda de contactos hecha en Python con interfaz en Tkinter. Los contactos se guardan en el archivo `datos/contactos.dat`, donde cada contacto ocupa exactamente 100 bytes. Como todos miden lo mismo se puede ir directo a cualquier contacto con `seek()` sin leer los que están antes.

Para buscar por id se usa un índice guardado en `indice.json`. La app también exporta los contactos a JSON y a un reporte PDF, revisa con SHA-256 si alguien modificó el archivo fuera del programa y crea respaldos en ZIP.

El código y el historial de commits están en el repositorio `https://github.com/Sebastian6918/Proyecto-II-Manejo`.

Cómo está organizado el código

Separé el código en módulos para que cada archivo se encargue de una sola cosa. La ventana no toca los archivos directamente, todo se lo pide a `servicio.py`.

Archivo	Para qué sirve
`main.py`	Abre la ventana.
`agenda/config.py`	Rutas de los archivos y largo de cada campo del registro.
`agenda/registro.py`	Clase Contacto: la convierte en 100 bytes y de vuelta, y valida los datos.
`agenda/archivo_contactos.py`	Acceso aleatorio al .dat con seek() y tell().
`agenda/indice.py`	Índice en JSON que relaciona cada id con su número de registro.
`agenda/integridad.py`	Hash SHA-256 del archivo de contactos.
`agenda/exportador.py`	Exporta a JSON y arma el PDF a partir del JSON.
`agenda/respaldo.py`	Crea el respaldo ZIP.
`agenda/metadatos.py`	Tamaño, fechas y permisos con os.stat y os.access.
`agenda/servicio.py`	Lógica de la agenda: une todo lo anterior.
`agenda/interfaz.py`	Ventana, menú, formulario y manejo de errores.
`cargar_datos.py`	Script que genera 40 contactos de prueba.

2. Estructura del registro

Cada contacto tiene 5 campos. A cada uno le asigné un largo fijo y, si el dato es más corto, se rellena con espacios hasta completarlo. Al final de cada registro va un salto de línea, para que el archivo se pueda abrir en el Bloc de notas y se vea un contacto por línea.

Campo	Largo (bytes)	Posición en el registro	Ejemplo
id	5	0 a 4	53
nombre	30	5 a 34	Gabriela Cifuentes
telefono	14	35 a 48	5178-6577
correo	32	49 a 80	gabriela.cifuentes96@url.edu.gt
ciudad	18	81 a 98	Sololá
salto de línea	1	99	\n
Total	100		

Los largos los escogí pensando en datos de acá. El teléfono tiene 14 para que quepa con código de país (+502 5555-1234), la ciudad tiene 18 para que quepan nombres como "Quetzaltenango" o "Antigua Guatemala", y el id tiene 5, así que el id más grande permitido es 99999. Todo esto está definido en `config.py`.

```
CAMPOS = (
    ("id", 5),
    ("nombre", 30),
    ("telefono", 14),
    ("correo", 32),
    ("ciudad", 18),
)
FIN_REGISTRO = b"\n"

# 5 + 30 + 14 + 32 + 18 + 1 = 100
TAMANO_REGISTRO = sum(largo for _, largo in CAMPOS) + len(FIN_REGISTRO)
```

Por qué se cuenta en bytes y no en letras

El archivo se guarda en UTF-8, y ahí las letras con tilde y la ñ ocupan 2 bytes. Si rellenara con `ljust()` contando letras, "Sololá" (6 letras) recibiría 12 espacios, pero en el archivo ocuparía 19 bytes en vez de 18. Ese registro mediría 101 y se correrían las posiciones de todos los que vienen después.

Por eso hice una función que hace lo mismo que `ljust()` pero midiendo bytes.

```
def rellenar(texto, largo):
    # como ljust() pero contando bytes, así el campo siempre mide lo mismo
    datos = texto.encode(CODIFICACION)
    if len(datos) > largo:
        datos = datos[:largo].decode(CODIFICACION, errors="ignore").encode(CODIFICACION)
    return datos.ljust(largo, b" ")
```

Antes de guardar, `Contacto.validar()` revisa que ningún campo pase su límite en bytes, entonces el recorte de la función no debería pasar nunca. Lo dejé solo por si acaso.

Este es el registro del contacto 53 dentro de `contactos.dat`, visto como bytes en Python. La á aparece como `\xc3\xa1`, que son sus 2 bytes.

```
b'53   Gabriela Cifuentes           5178-6577     gabriela.cifuentes96@url.edu.gt Solol\
xc3\xa1           \n'
```

Si se abre el .dat en el Bloc de notas, las ciudades con tilde se ven corridas un espacio a la izquierda (se puede ver en la figura 6). En pantalla la á se ve como una letra, pero en el archivo son 2 bytes, por eso lleva un espacio menos de relleno.

3. Acceso aleatorio con seek() y tell()

Como todos los registros miden 100 bytes, la posición de cualquier contacto se calcula multiplicando.

posición = número de registro x 100

El registro 0 empieza en el byte 0, el 1 en el byte 100, el 4 en el byte 400, y así. Con `seek()` se mueve el puntero del archivo directo a esa posición y con `read(100)` se lee solo ese contacto. `tell()` devuelve en qué byte quedó el puntero, y lo uso para comprobar que el salto llegó donde tenía que llegar y para mostrarlo en la ventana.

El archivo se abre en modo binario (`rb`, `r+b`, `ab`). En modo texto, en Windows, `seek()` no garantiza posiciones exactas por los saltos de línea y los caracteres de varios bytes. En binario un byte es un byte, y el contenido igual es texto UTF-8: se codifica al escribir y se decodifica al leer.

Leer un contacto

```
def leer(self, numero_registro):
    # aquí está el acceso aleatorio: salta directo al registro sin leer los de antes
    if not 0 <= numero_registro < self.total_registros():
        raise IndexError(f"El registro #{numero_registro} no existe.")
    with open(self.ruta, "rb") as f:
        f.seek(self.posicion_de(numero_registro))  # numero_registro * 100
        posicion = f.tell()
        datos = f.read(TAMANO_REGISTRO)
    return ResultadoAcceso(numero_registro, posicion, Contacto.desde_bytes(datos))
```

Modificar en el lugar

Para modificar se abre con `r+b`, que permite leer y escribir sin borrar el archivo (`w` lo dejaría vacío). Se hace `seek()` al registro y se escriben encima solo esos 100 bytes; el resto del archivo no se toca.

```
def modificar(self, numero_registro, contacto):
    # solo se sobrescriben esos 100 bytes, el resto del archivo ni se toca
    datos = contacto.a_bytes()
    with open(self.ruta, "r+b") as f:  # r+b no borra el archivo como haría w
        f.seek(self.posicion_de(numero_registro))
        posicion = f.tell()
        f.write(datos)
    return ResultadoAcceso(numero_registro, posicion, contacto)
```

Agregar al final

Para agregar se va al final con `seek(0, os.SEEK_END)` y `tell()` dice en qué byte va a quedar el nuevo contacto. Si esa posición no es múltiplo de 100, el archivo está dañado y no se agrega nada, para no empeorarlo.

```
with open(self.ruta, "ab") as f:
    f.seek(0, os.SEEK_END)  # al final del archivo
    posicion = f.tell()
    if posicion % TAMANO_REGISTRO != 0:
        raise ArchivoCorruptoError(...)
    f.write(datos)
return ResultadoAcceso(posicion // TAMANO_REGISTRO, posicion, contacto)
```

Pruebas en la aplicación

En la ventana hay un panel que muestra el seek y el tell de cada operación, y la bitácora de abajo los va anotando. En la figura 1 se buscó el id 53, el índice dice que está en el registro 4, se hizo `seek(400)` y `tell()` devolvió 400.

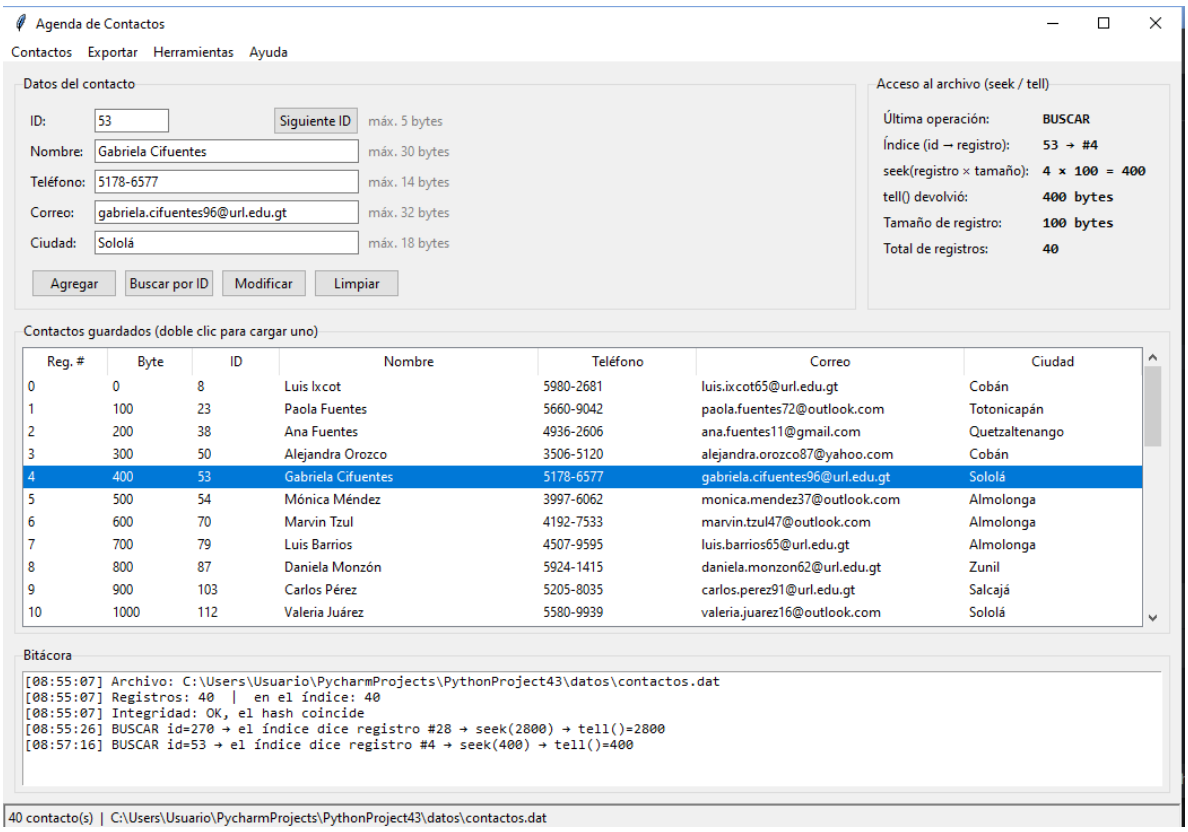


Figura 1. Búsqueda del id 53. Registro 4, seek(400) y tell() = 400.

En la figura 2, al agregar un contacto nuevo, quedó en el registro 40 porque ya había 40 contactos (del 0 al 39), y `tell()` dio 4000. En la figura 3 se modificó la ciudad del id 79: se escribieron solo los 100 bytes que empiezan en el byte 700.

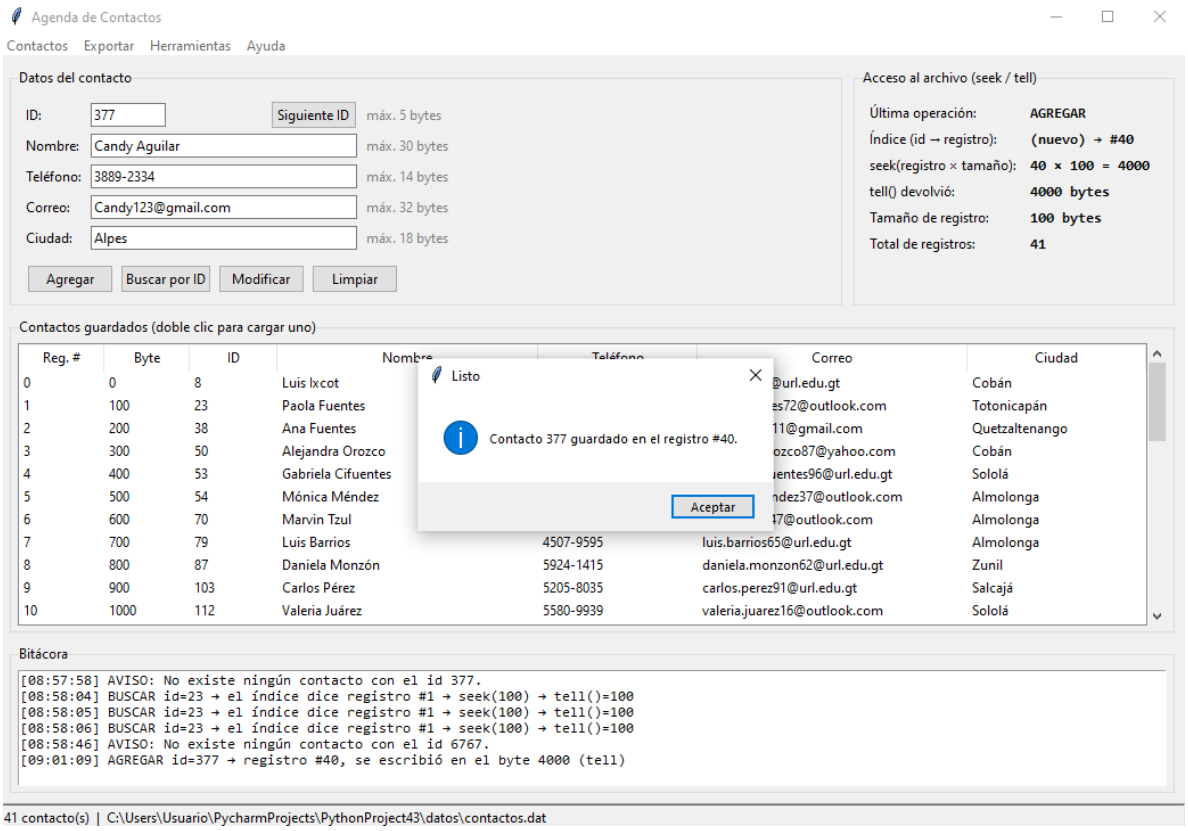


Figura 2. Se agregó el contacto 377 en el registro 40 (byte 4000).

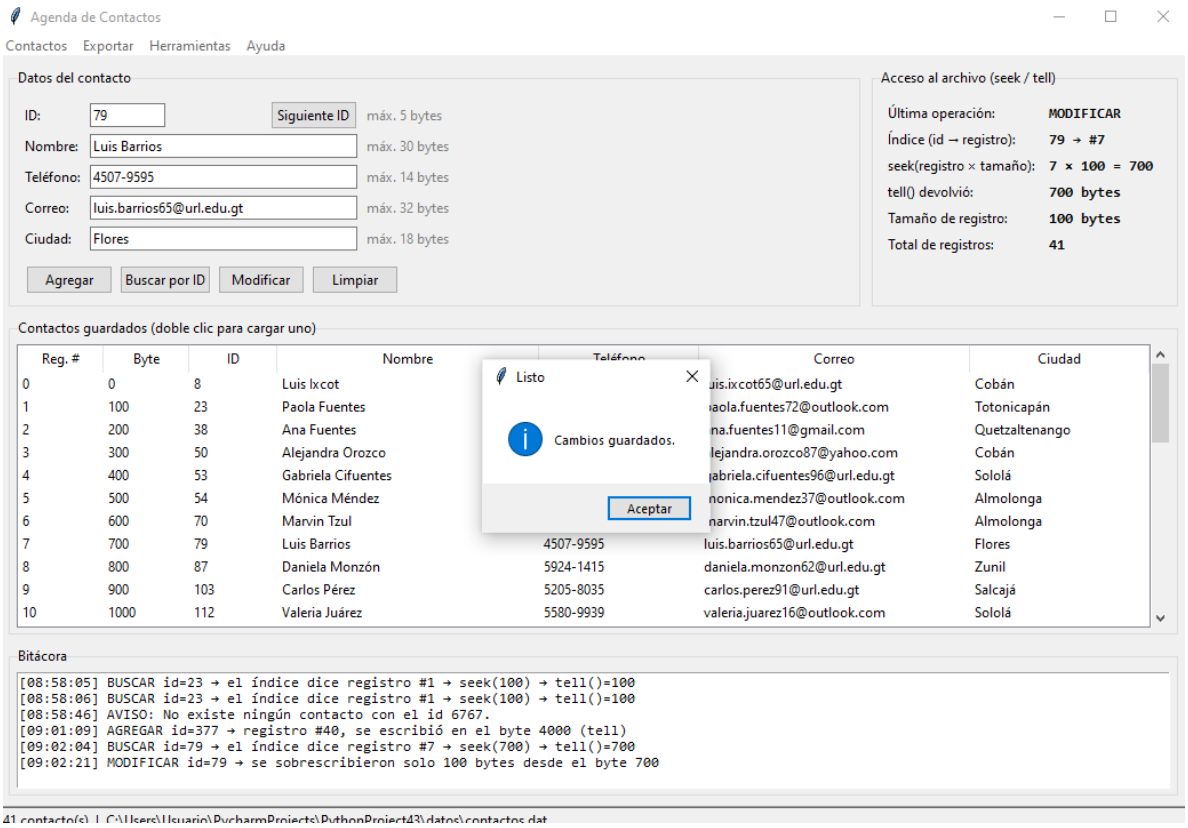


Figura 3. Modificación del registro 7, solo se escribieron 100 bytes desde el byte 700.

4. Índice primario

El índice es un diccionario que dice en qué número de registro está cada id. Se guarda en `datos/indice.json` y así se ven sus primeras entradas con los datos de prueba:

```
{
  "8": 0,
  "23": 1,
  "38": 2,
  "50": 3,
  "53": 4,
  ...
}
```

Los ids de prueba van saltados a propósito (8, 23, 38...) para que se note que el id no es lo mismo que la posición. El id 23 está en el registro 1 y no en el 23, y para saberlo sin leer el archivo se necesita el índice.

Cómo se busca

Primero se consulta el índice. Si el id no está se avisa que no existe, y si está se hace un solo `seek()` al registro.

```
def buscar(self, id_contacto):
    # primero el índice, luego seek directo al registro
    numero = self.indice.buscar(id_contacto)
    if numero is None:
        raise IdNoEncontradoError(f"No existe ningún contacto con el id {id_contacto}.")
    return self.archivo.leer(numero)
```

El índice se actualiza cada vez que se agrega un contacto (`Indice.agregar` guarda el JSON en ese momento). Al modificar no hace falta porque el contacto sigue en el mismo registro. Si `indice.json` se

borra, se daña o no cuadra con el .dat, al abrir la app se reconstruye leyendo el .dat de corrido, entonces no se pierde nada.

Para guardarlo primero se escribe un archivo `.tmp` y después se reemplaza el original con `os.replace()`, para que si algo falla a medio guardar no se pierda el índice que ya estaba.

Por qué es más eficiente que una búsqueda secuencial

Sin índice, para encontrar un id habría que leer el registro 0, ver si es ese id, luego el 1, y así hasta encontrarlo. En el peor caso se lee todo el archivo. Con el índice, buscar en el diccionario tarda lo mismo sin importar cuántos contactos haya y después se lee un solo registro.

Caso	Búsqueda secuencial	Búsqueda con índice
Buscar el último contacto (id 377, registro 40 de 41)	41 lecturas (4,100 bytes)	1 lectura (100 bytes)
Buscar un id que no existe, con 41 contactos	41 lecturas, hasta el final	Ninguna, el índice ya sabe que no está
Agenda con 10,000 contactos, peor caso	10,000 lecturas (1 MB)	1 lectura (100 bytes)
En promedio, con n contactos	n/2 lecturas, O(n)	1 lectura, O(1)

Con 41 contactos la diferencia casi no se siente, pero la búsqueda secuencial crece junto con el archivo y la del índice no. Lo que cuesta es mantener el `indice.json` actualizado y el espacio extra que ocupa (557 bytes en este caso).

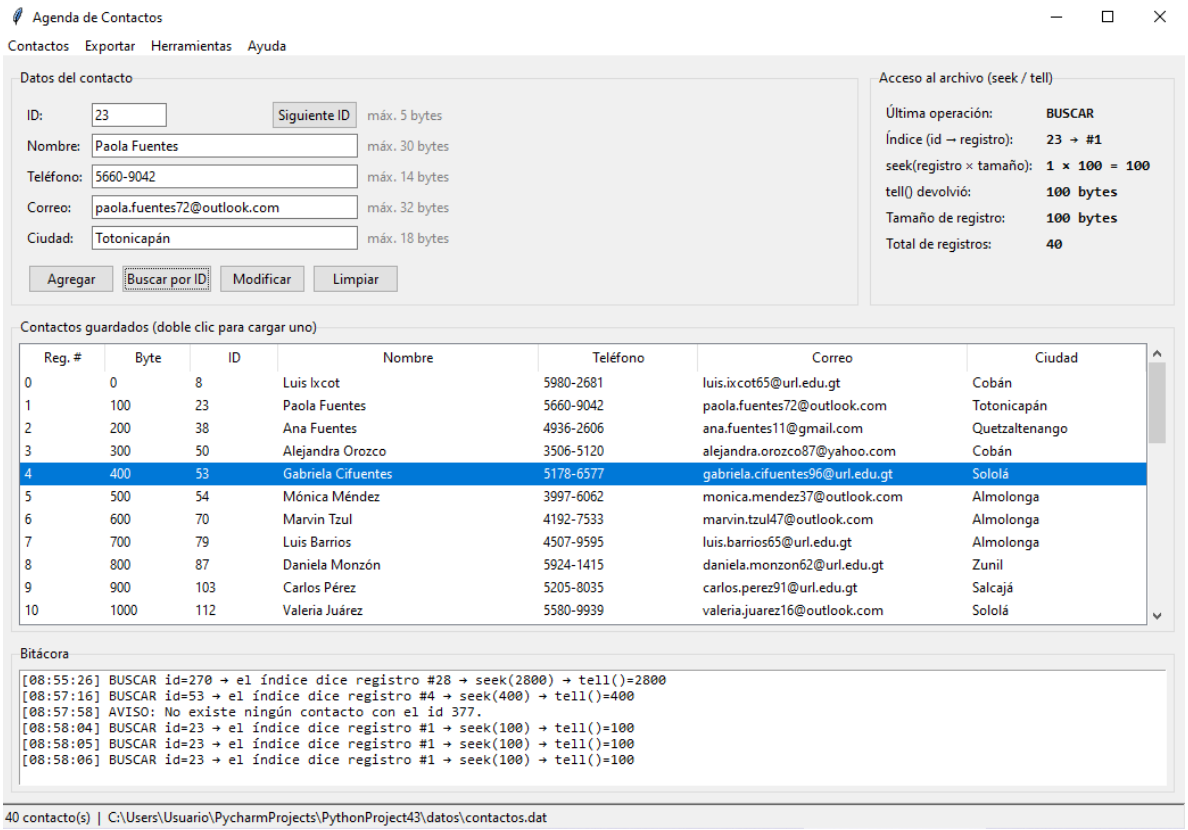


Figura 4. El id 23 está en el registro 1. seek(100) y tell() = 100.

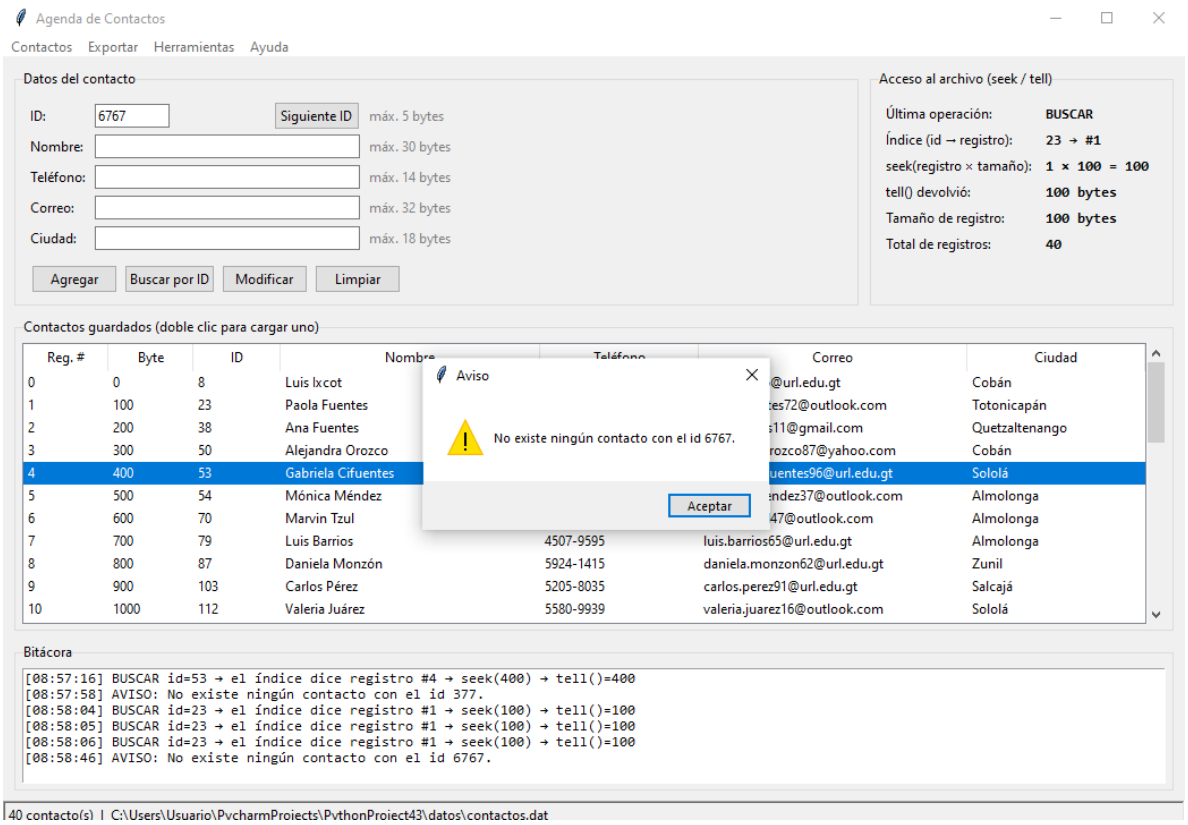


Figura 5. Búsqueda de un id que no existe (6767).

5. Verificación de integridad con SHA-256

Cada vez que la app guarda un cambio (agregar o modificar), calcula el hash SHA-256 de `contactos.dat` con `hashlib` y lo guarda en `contactos.sha256`, con el mismo formato que usa el comando `sha256sum`. Al abrir la app se vuelve a calcular y se compara con el guardado. Si alguien cambió aunque sea un byte por fuera, el hash sale completamente distinto.

```
def calcular_sha256(ruta):
    sha = hashlib.sha256()
    with open(ruta, "rb") as f:
        # se lee por pedazos de 4 KB para no cargar todo el archivo en memoria
        for bloque in iter(lambda: f.read(4096), b""):
            sha.update(bloque)
    return sha.hexdigest()
```

Si los hashes no coinciden sale una alerta con los dos hashes que pregunta si se acepta el archivo como está. Si se dice que sí, se guarda el hash nuevo y se reconstruye el índice por si el cambio movió algún id. Si se dice que no, la alerta sigue saliendo cada vez que se abre la app.

Prueba con el archivo alterado a mano

Con la app cerrada abrí `contactos.dat` en el Bloc de notas y cambié el nombre del contacto 38 de Ana Fuentes a Pau Fuentes. Cambié 3 letras por otras 3 sin tilde para que el tamaño del registro no cambiara.

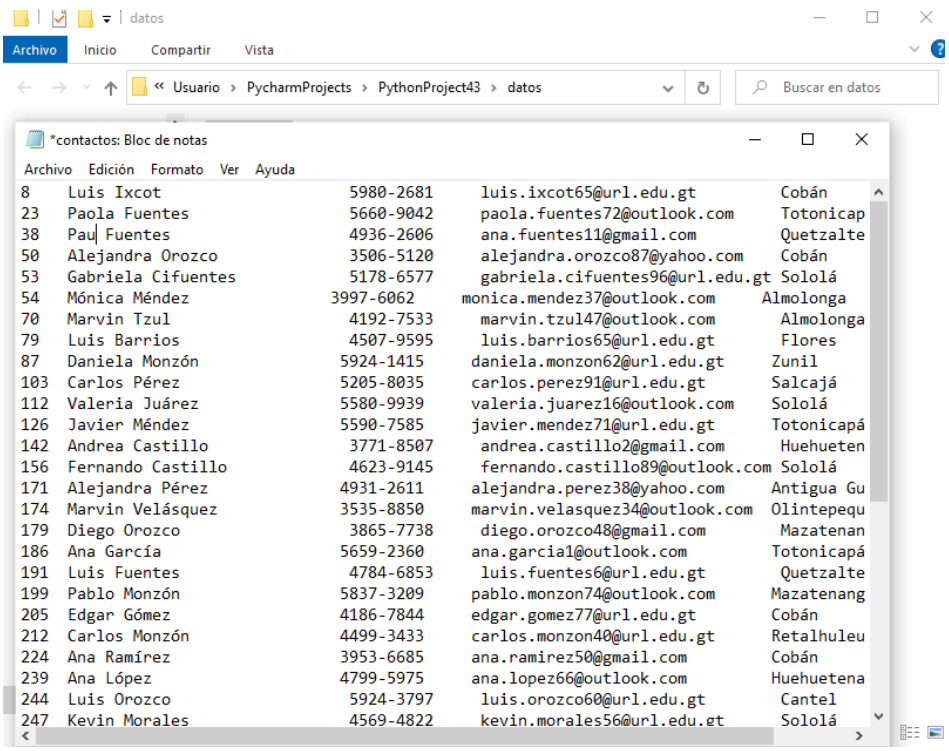


Figura 6. Cambio hecho a mano en el Bloc de notas, de Ana Fuentes a Pau Fuentes.

Al abrir la app el hash ya no coincidía y salió la alerta.

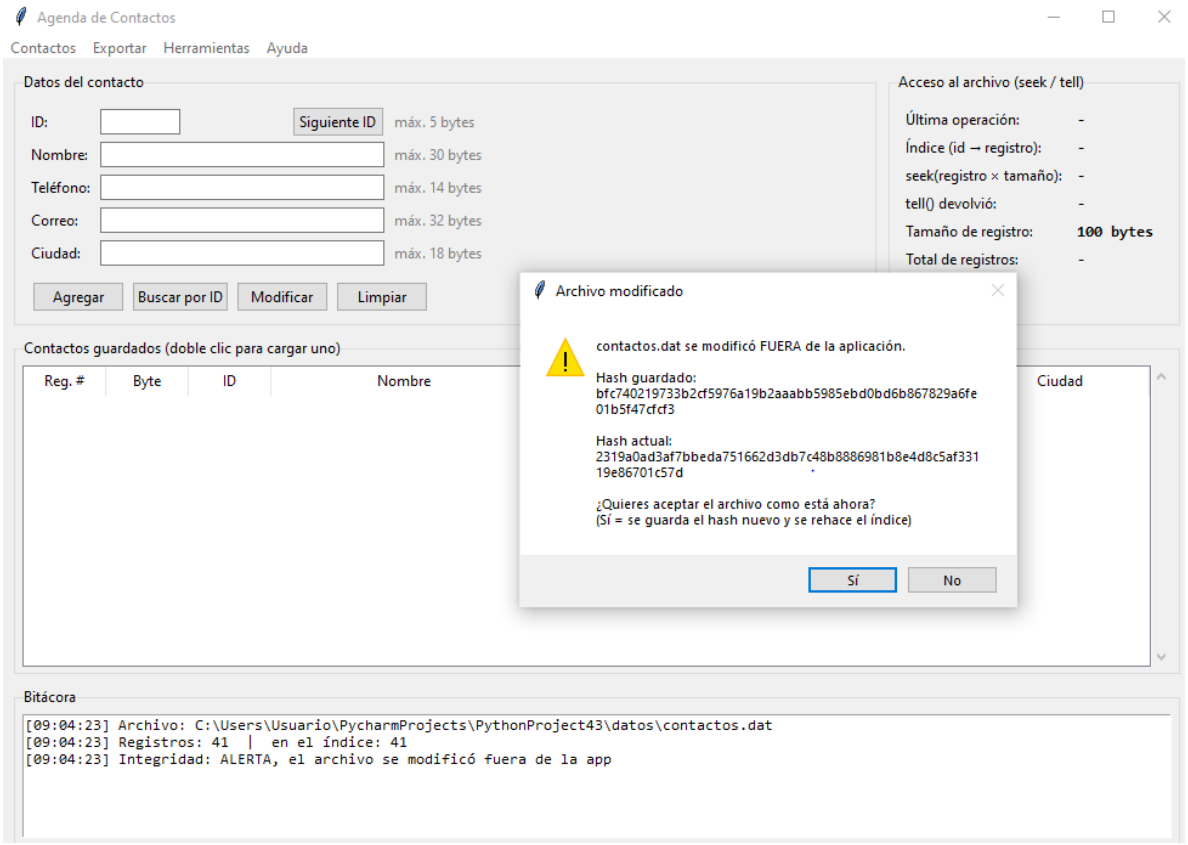


Figura 7. Alerta al abrir la aplicación después del cambio.

```
Guardado: bfc740219733b2cf5976a19b2aaabb5985ebd0bd6b867829a6fe01b5f47cfcf3
Al abrir: 2319a0ad3af7bbeda751662d3db7c48b8886981b8e4d8c5af33119e86701c57d
```

Le di Sí. Después, en Herramientas > Verificar integridad, ya sale que está bien con el hash nuevo (`2319a0ad...`), y en la tabla aparece Pau Fuentes en el registro 2.

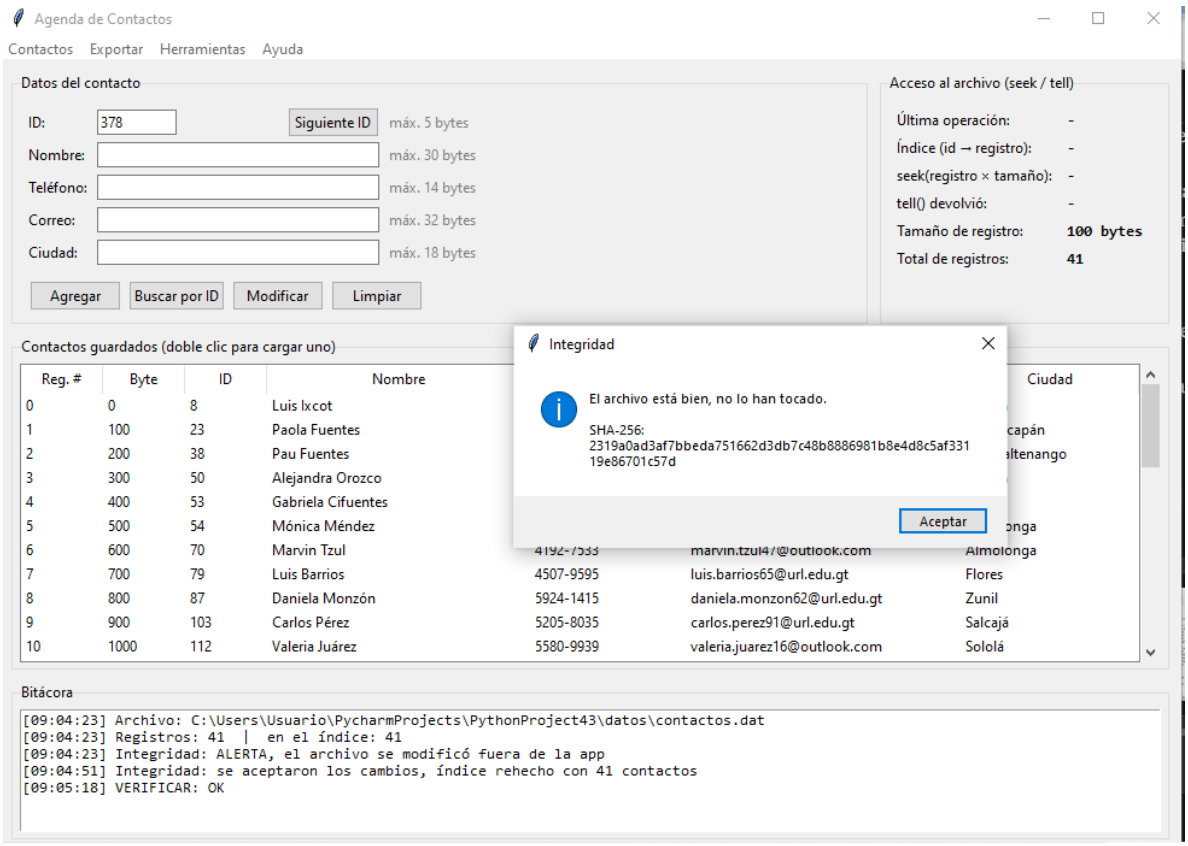


Figura 8. Verificación de integridad después de aceptar el cambio.

Un problema que tuve al hacer la prueba

La primera vez que intenté la prueba edité el .dat dentro de PyCharm. Al guardar, PyCharm le borró los espacios del final a la línea que cambié y el archivo pasó de 200 a 188 bytes. La app avisó que el tamaño ya no era múltiplo de 100 y que el archivo parecía dañado, pero no se cayó, leyó lo que pudo y no dejó agregar más contactos. Con eso me di cuenta de lo delicado que es un archivo de registros de tamaño fijo, porque con quitar unos espacios ya se corren todas las posiciones. La prueba final la hice con el Bloc de notas, que no toca los espacios.

6. Exportación a JSON y reporte PDF

La opción Exportar > Exportar a JSON lee todos los contactos y los guarda en `exportados/contactos.json`, junto con la fecha y el total. Se usa `ensure_ascii=False` para que las tildes queden escritas normal y no como `\u00e9`.

```
{
  "generado": "2026-11-09T09:05:59",
  "total": 41,
  "contactos": [
    {
      "id": 8,
      "nombre": "Luis Ixcot",
      "telefono": "5980-2681",
      "correo": "luis.ixcot65@url.edu.gt",
      "ciudad": "Cobán"
    },
    ...
```

El reporte PDF se arma a partir de ese JSON y no del .dat. La opción Generar reporte PDF primero exporta el JSON actualizado, y después `generar_pdf_desde_json()` lo lee y arma una tabla con fpdf2. El reporte

tiene la fecha de generación, el total de contactos y el número de página. Si la tabla no cabe en una hoja se crea otra y se repite el encabezado.

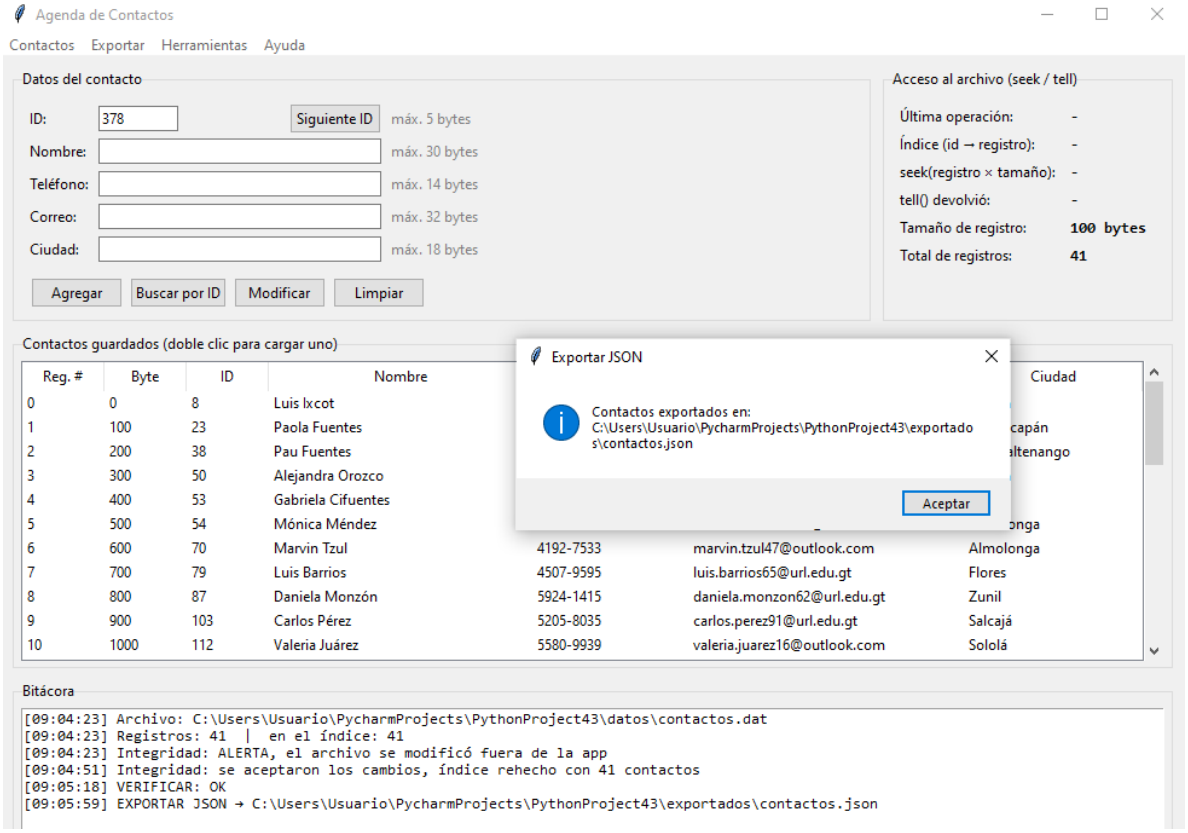


Figura 9. Exportación a JSON.

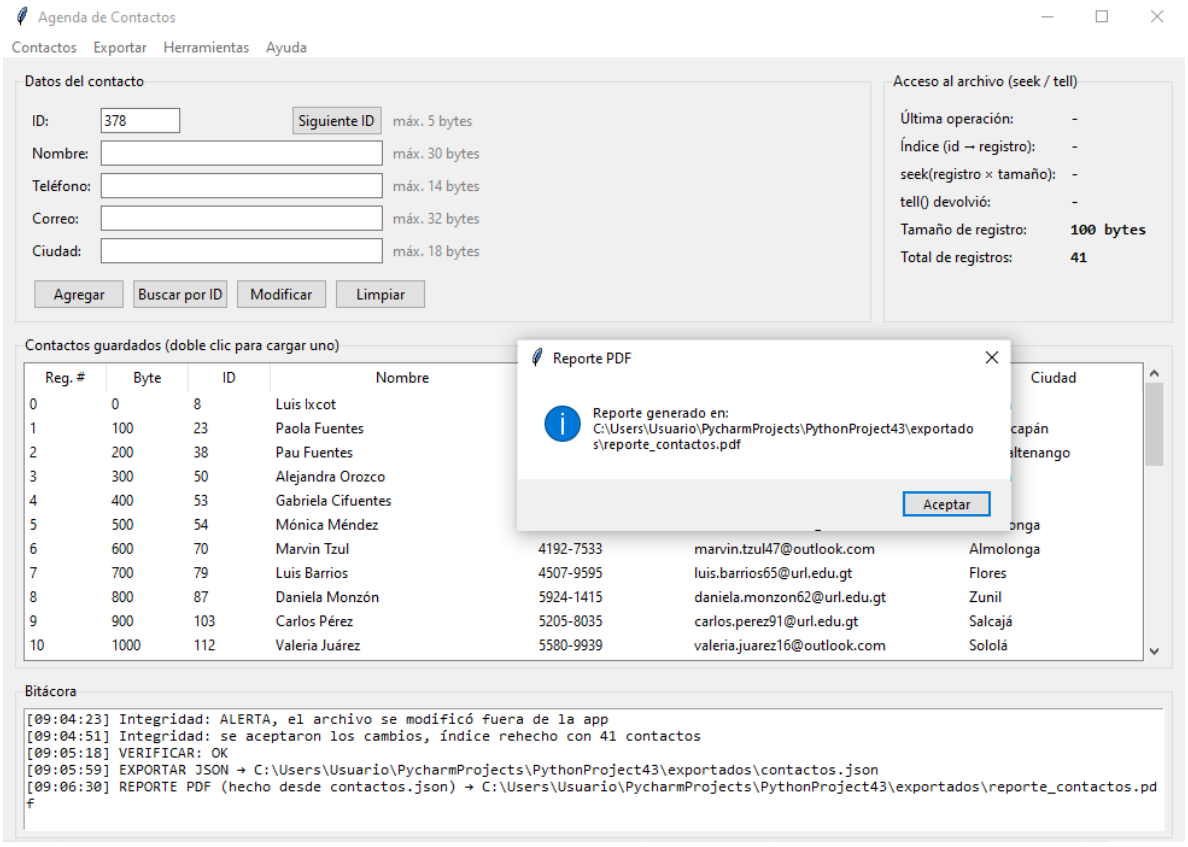


Figura 10. Generación del reporte PDF.

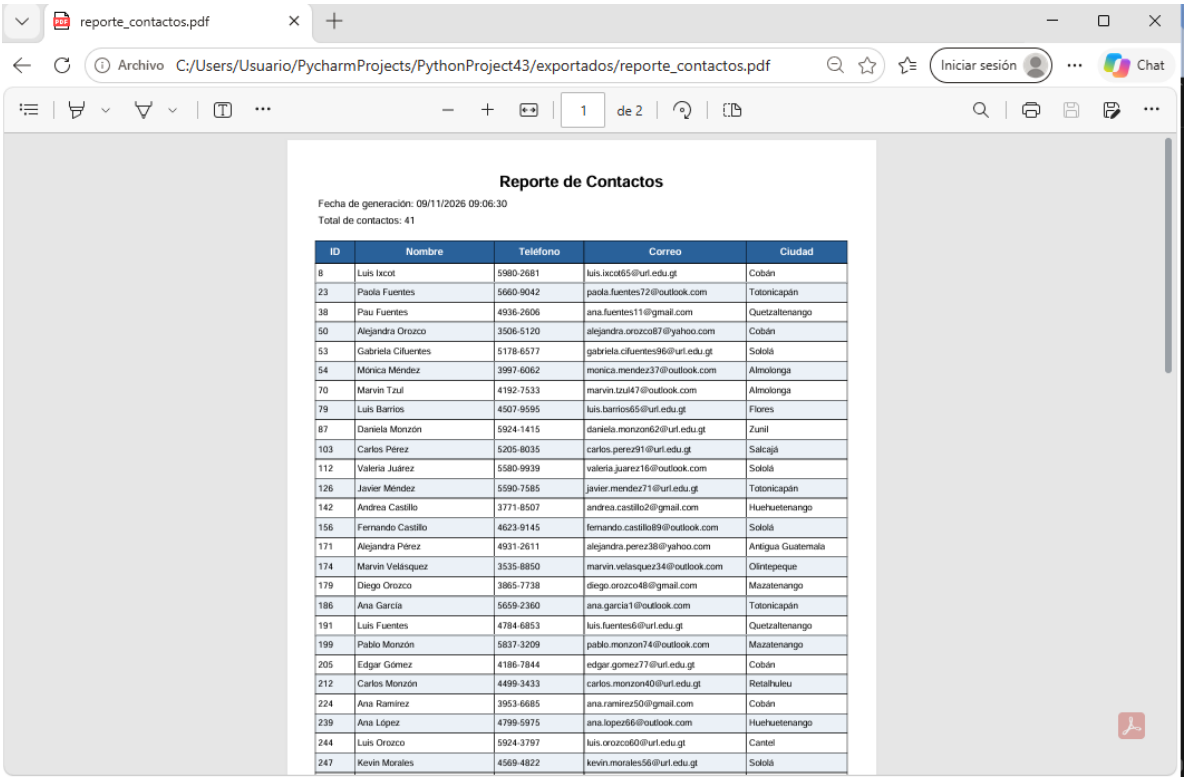


Figura 11. Reporte PDF generado (2 páginas con 41 contactos).

7. Respaldo comprimido en ZIP

La opción Herramientas > Crear respaldo ZIP mete `contactos.dat` e `indice.json` en un archivo `respaldos/respaldo_AAAAMMDD_HHMMSS.zip`, con compresión `ZIP_DEFLATED` al nivel máximo (9). Como el nombre lleva la fecha y la hora, cada respaldo es un archivo nuevo y no reemplaza al anterior. Al terminar se muestra el tamaño original y el comprimido.

Archivo	Tamaño original	Comprimido dentro del ZIP
contactos.dat	4,100 bytes (4.00 KB)	1.29 KB
indice.json	557 bytes	200 bytes
Total	4,657 bytes (4.55 KB)	ZIP completo: 1,748 bytes (1.71 KB)

El respaldo pesa 62.5 % menos que los archivos originales. Comprime bastante porque los registros tienen muchos espacios de relleno seguidos, y la compresión funciona mejor con lo que se repite.

En las pruebas con solo 2 o 3 contactos el ZIP salía más pesado que los originales. Eso pasa porque el ZIP le agrega a cada archivo una cabecera con su nombre, fecha y otros datos, de unos 100 bytes. Con pocos datos esa cabecera pesa más de lo que se ahorra, pero con 41 contactos ya no se nota.

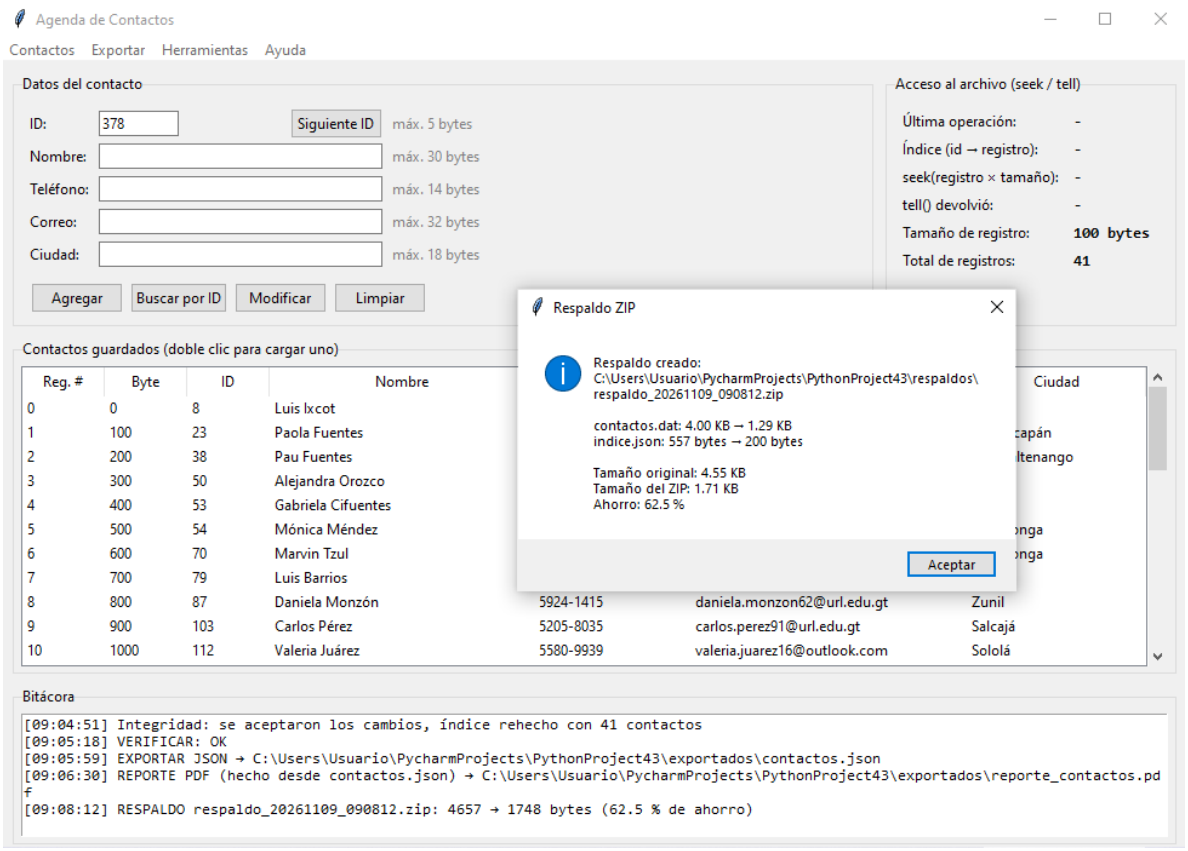


Figura 12. Respaldo ZIP con el tamaño original y el comprimido.

8. Metadatos y permisos

La ventana Herramientas > Información del archivo muestra los datos de `contactos.dat`, `indice.json` y `contactos.sha256`. Para eso se usan dos funciones del módulo `os`.

`os.stat()` da el tamaño (`st_size`), la fecha de última modificación (`st_mtime`) y en Windows la fecha de creación (`st_ctime`). Las fechas vienen en segundos y se pasan a fecha normal con `datetime.fromtimestamp()`.

`os.access()` revisa si el usuario que corre el programa puede leer (`os.R_OK`) y escribir (`os.W_OK`) el archivo. Si el archivo se pone como solo lectura, el permiso de escritura sale en No.

En la figura 13 se ve que `contactos.dat` mide 4,100 bytes, que son 41 registros de 100 bytes.

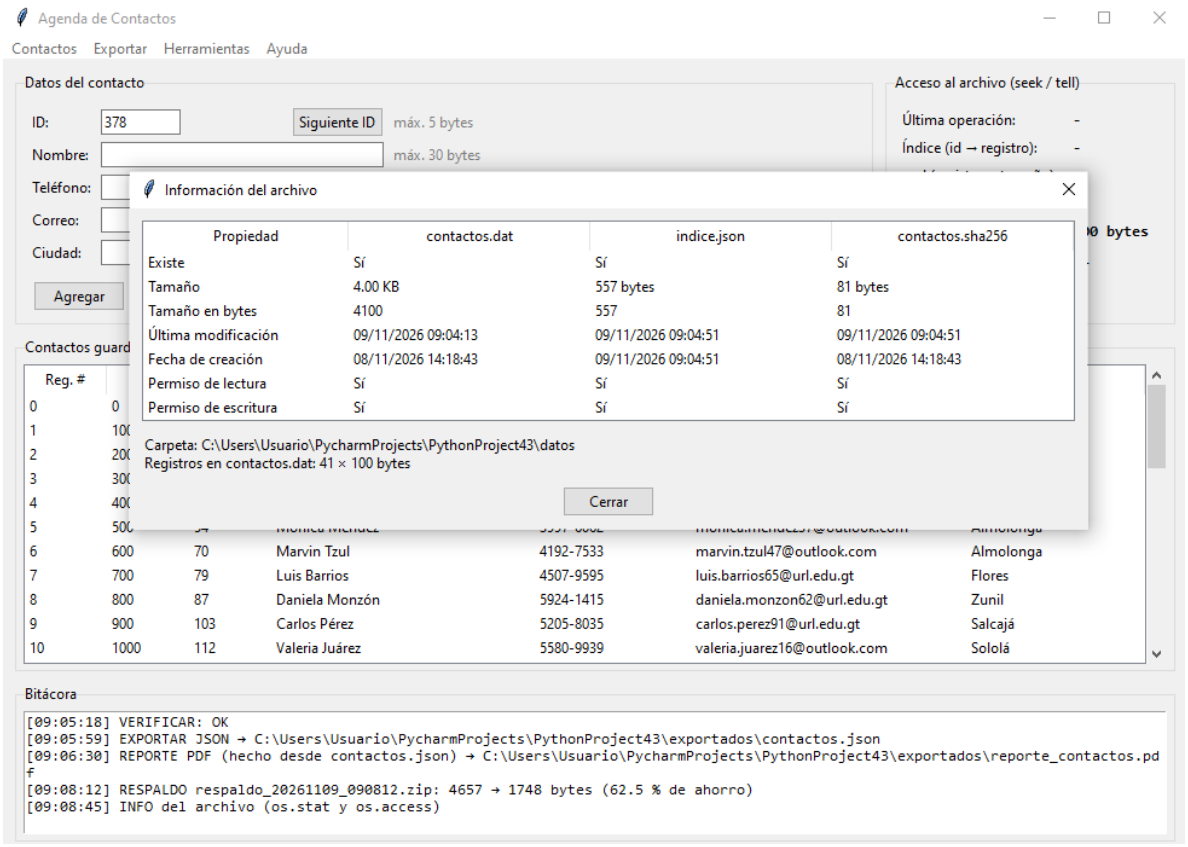


Figura 13. Información del archivo: tamaño, fechas y permisos.

9. Manejo de errores

Cada acción de la ventana tiene el decorador `@manejar_errores`, que envuelve la acción en un `try/except`. Con eso no hay que repetir el mismo bloque en todos los botones y ningún error cierra la aplicación. Según el tipo de error se muestra un mensaje distinto y se anota en la bitácora.

```
def manejar_errores(metodo):
    @functools.wraps(metodo)
    def envoltura(self, *args, **kwargs):
        try:
            return metodo(self, *args, **kwargs)
        except ContactoInvalidoError as e:
            messagebox.showwarning("Datos inválidos", str(e), parent=self)
        except AgendaError as e:            # id que no existe, id repetido
            ...
        except PermissionError as e:        # sin permisos sobre el archivo
            ...
        except OSError as e:                # cualquier otro error de archivo
            ...
    return envoltura
```

Situación	Qué hace la aplicación
No existe `contactos.dat`	Lo crea vacío al abrir (también las carpetas).
No existe `indice.json`	Lo reconstruye leyendo el .dat.
`indice.json` dañado (no es JSON válido)	Avisa y lo reconstruye desde el .dat.
El índice no cuadra con el .dat	Avisa y lo reconstruye.
El tamaño del .dat no es múltiplo de 100	Avisa que está dañado, lee solo los registros completos y no deja agregar.

Situación	Qué hace la aplicación
Un registro no se puede leer	Lo salta, lo anota en la bitácora y sigue con los demás.
El .dat se modificó fuera de la app	Muestra la alerta con los dos hashes (sección 5).
Sin permisos de lectura o escritura	Muestra un mensaje de que no hay permisos y la app sigue abierta.
Se busca un id que no existe	Avisa que no existe ningún contacto con ese id.
Se agrega un id que ya existe	Avisa que el id ya existe y no lo guarda.
Datos inválidos (correo sin @, campo vacío o muy largo)	Muestra todos los problemas encontrados y no lo guarda.

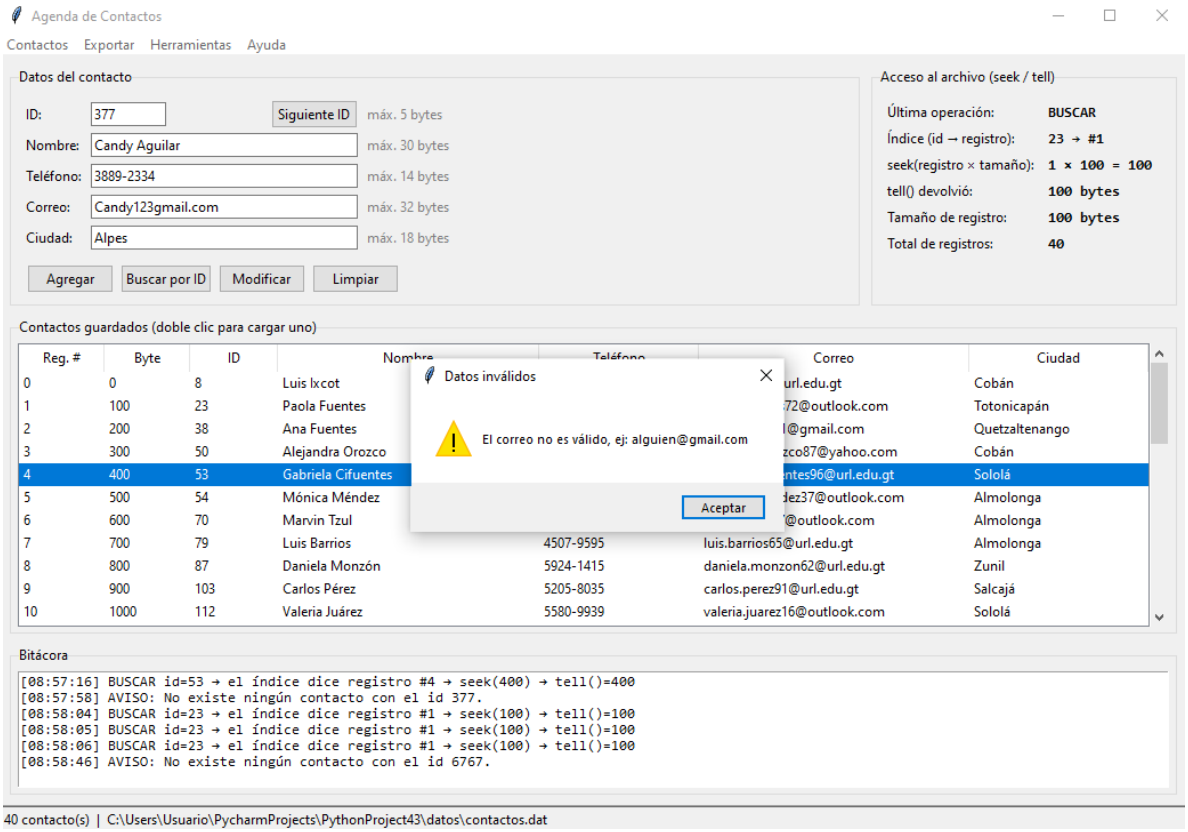


Figura 14. Validación del correo sin @.

10. Conclusiones

Con los registros de tamaño fijo se puede llegar a cualquier contacto con una multiplicación y un seek(), sin leer el resto del archivo. Para que funcione hay que medir los campos en bytes y no en letras, porque en UTF-8 las tildes ocupan 2 bytes y un solo registro de más o de menos corre todas las posiciones.

El índice en JSON es lo que permite buscar por id de forma directa, porque los ids no tienen por qué coincidir con la posición en el archivo. Si se pierde, se puede volver a armar desde el .dat.

El hash SHA-256 sirvió para detectar el cambio que hice a mano en el archivo, aunque fueran solo 3 letras. El respaldo ZIP redujo el tamaño en 62.5 % porque los espacios de relleno se comprimen muy bien.